

# Informe de Reflexión — Examen Final CI/CD

nnyez — 29 de julio de 2026 — <https://github.com/nnyez/inventario-app>

## 1. Justificación de la estrategia Canary

Se eligió Canary sobre Blue-Green porque Kubernetes distribuye tráfico proporcional a la cantidad de pods que matchean el selector del Service. Con 4 réplicas estables (v1/blue) y 1 canary (v2/green), el  $\sim 80\%$  del tráfico va a la versión estable y  $\sim 20\%$  a la nueva. Canary requiere solo 5 pods frente a 8 de Blue-Green, es más eficiente para una aplicación liviana como `inventario-app`, y permite rollback inmediato escalando el canary a 0 réplicas. Además, permite aumentar gradualmente el porcentaje de tráfico a la nueva versión simplemente escalando el número de réplicas canary.

## 2. Observación sobre persistencia de datos

Se creó un producto vía API (4 productos total) y se forzó la eliminación de un pod. Tras la recreación, el catálogo contenía solo 3 productos (los seed originales). Esto ocurre porque la base de datos es un archivo JSON local (`data/products.json`) en el filesystem efímero del pod. No es un error, es el comportamiento esperado sin PVC o base de datos externa.

## 3. Problemas reales encontrados

| Problema                    | Causa                                                     | Solución                            |
|-----------------------------|-----------------------------------------------------------|-------------------------------------|
| Pipeline fallaba            | <code>trivy-action@0.28.0</code> no existe                | Cambiar a <code>@v0.36.0</code>     |
| Build fallaba por caché     | <code>cache-to=type=gha</code> incompatible               | Eliminar caché                      |
| Error “denied: not allowed” | <code>GITHUB_TOKEN</code> sin <code>packages:write</code> | Agregar permissions                 |
| Trivy CRITICALs             | <code>node:18-alpine</code> con CVE OpenSSL/tar           | Alpine 3.21, apk upgrade            |
| Pods no arrancaban          | Secret no existía                                         | <code>kubectrl create secret</code> |
| Sin balanceo port-forward   | <code>port-forward</code> no distribuye tráfico           | Probar dentro del clúster           |

## 4. Métricas DORA

Se calcularon con datos verificables de tres fuentes: `git log` (timestamp de commits), `gh run list` (timestamp de pipelines), y `kubectrl describe deployment` (campo `restartedAt` para el momento exacto en que el cambio llegó al clúster).

| Métrica                                                     | Valor           | Nivel DORA   |
|-------------------------------------------------------------|-----------------|--------------|
| <i>Lead Time</i> (commit → clúster):                        | 16 seg – 41 min | <b>Élite</b> |
| Despliegue inicial: 3ab1521 04:23:30 → 05:04:29             | 41 min          | (<1 h)       |
| Namespace+secrets: 112578a 05:06:48 → 05:07:04              | 16 seg          |              |
| STARTUP_DELAY: 2d25c12 05:07:51 → ~05:08                    | ~1 min          |              |
| <i>Deployment Frequency</i> : cambios promovidos al clúster | 5 deploys/día   | <b>Alto</b>  |
| <i>Change Failure Rate</i> : pipelines fallidos / total     | 7/14 = 50 %     | <b>Bajo</b>  |

## 5. Reflexión

El CFR del 50 % refleja la etapa inicial de prueba y error (7 fallos por autenticación, versión incorrecta de acciones, caché de Docker y vulnerabilidades CRITICAL detectadas por Trivy). En un entorno maduro se espera reducirlo por debajo del 15 % (nivel Alto/Élite). El lead time **Élite** (16 segundos para cambios pequeños como configuración de namespace o STARTUP\_DELAY) demuestra que el pipeline entrega cambios casi instantáneamente cuando está estable. La frecuencia **Alta** (5 deploys/día) es propia de un desarrollo iterativo donde se experimenta y corrige sobre infraestructura real. En conjunto, las métricas confirman que las prácticas implementadas (fail-fast entre jobs, escaneo de seguridad con Trivy, readiness probe tolerante al arranque lento, y manejo de secretos vía `secretKeyRef`) mejoran significativamente la calidad y confiabilidad del despliegue.